

# **Computational Physics: Problem Solving with Julia**

Colton Quirk

2026-05-01

# Table of contents

|                                                            |          |
|------------------------------------------------------------|----------|
| <b>Preface</b>                                             | <b>3</b> |
| <b>1 Introduction</b>                                      | <b>4</b> |
| <b>2 Differential Equations and Nonlinear Oscillations</b> | <b>5</b> |
| 2.1 Nonlinear Oscillations . . . . .                       | 5        |
| 2.2 Orbit Around a Point Mass . . . . .                    | 5        |
| <b>3 Summary</b>                                           | <b>8</b> |
| <b>References</b>                                          | <b>9</b> |

# Preface

The idea for this book came from my love and appretiation of the book *Compuational Physics Problem Solving with Python* by Landau et al. (2024) and the Julia Programming Language.

# 1 Introduction

Computational Physics is a fascinating field which the Julia Programming Language is particularly well suited for.

## 2 Differential Equations and Nonlinear Oscillations

Numerical methods for solving ordinary differential equations. Applying those methods to nonlinear systems.

```
using Pkg
Pkg.activate("../")
using CairoMakie, LaTeXStrings, LinearAlgebra
```

Activating project at `~/Desktop/comp\_phys\_julia/chapters`

### 2.1 Nonlinear Oscillations

$$F_k(x) + F_{\text{ext}}(x, t) = m \frac{d^2 x}{dt^2} \quad (2.1)$$

### 2.2 Orbit Around a Point Mass

```
const G = 1.0
const M = 1.0;
```

```
function f(t, state)
    pos, vel = state
    acc = - G * M * pos / norm(pos)^3
    return [vel, acc]
end;
```

```
mutable struct Star{T}
    state::Array{T}
    pos_hist
end;
```

```

function rk4(t, y, force, dt)
    """
    Function to calculate the next timestep using:
        4th-order Runge-Kutta
    """
    k1 = force(t, y)
    k2 = force(t + 0.5 * dt, y + 0.5 * k1 * dt)
    k3 = force(t + 0.5 * dt, y + 0.5 * k2 * dt)
    k4 = force(t + dt, y + k3 * dt)

    y_new = y + dt * (k1 + 2.0*k2 + 2.0*k3 + k4) / 6.0
end;

```

```

function orbit(t_start, t_end, dt, star, force)
    t = t_start
    while t < t_end
        new_state = rk4(t, star.state, force, dt)
        star.state = new_state
        push!(star.pos_hist, star.state[1])
        t += dt
    end
    star.pos_hist = reduce(hcat, star.pos_hist)'
end;

```

```

start_pos = [1.0, 0.0, 0.0]
start_vel = [0.0, 1.0, 0.0]
start_state = [start_pos, start_vel]
star = Star(start_state, [start_state[1]])

h = 1e-4
norbits = 1.0
t_end = 2 * pi * norbits
orbit(0.0, t_end, h, star, f);

```

```

function plot_orbit(star, name; skip=5)
    inch = 96
    pt = 4/3

    fig = Figure(size=(10inch, 7inch), fontsize=16pt)
    ax = Axis(fig[1, 1],
        xlabel = "x",

```

```

        ylabel = "y",
        title=L"Orbit around a Point Mass ( $\Delta t = \%(h)$ )",
        aspect = DataAspect())

    lines!(ax, star.pos_hist[begin:skip:end, 1], star.pos_hist[begin:skip:end, 2])
    save(name, fig)
end;

with_theme(theme_latexfonts()) do
    plot_orbit(star, "images/rk4.svg"; skip=100)
end;

```

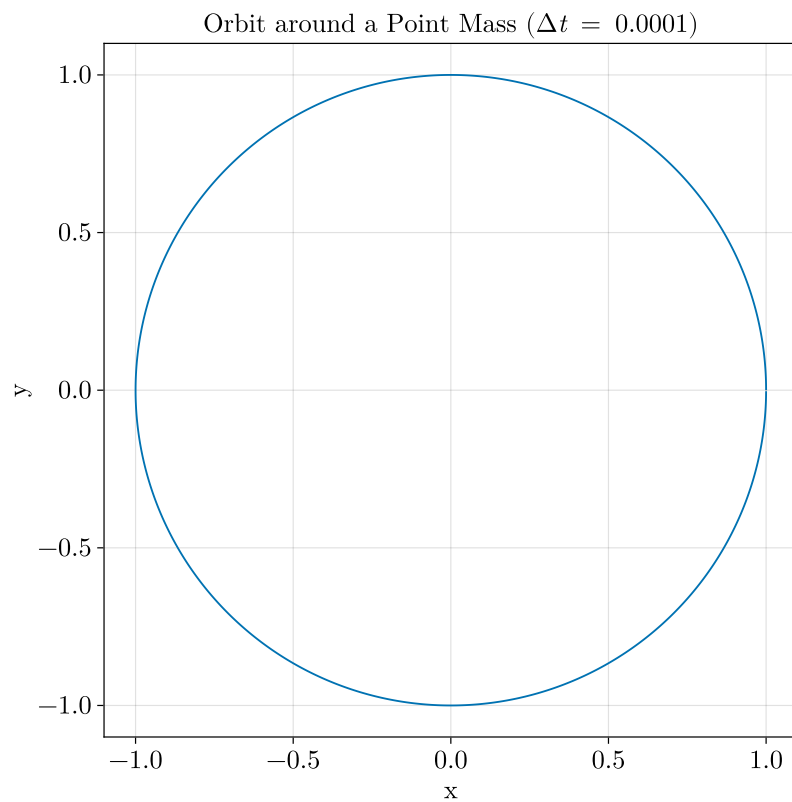


Figure 2.1: Orbit around a point mass using RK4

## 3 Summary

We have covered only a small fraction of what is possible with Julia in Computational Physics. I hope this book has inspired you to go on and solve physics problems with your computer on your own.



# References

Landau, Rubin H., Manuel J. Páez, and Cristian C. Bordeianu. 2024. *Computational Physics: Problem Solving with Python*. 4th ed. WILEY VCH.